

Tema 7 - Gestión de errores

Laboratorio Guiado

Objetivos	Practicar la gestión de errores en Python: errores de ejecución, <code>try/except</code> , captura específica, <code>else</code> , <code>finally</code> , <code>raise</code> , validación de datos, excepciones personalizadas, relanzamiento y diagnóstico básico con <code>traceback</code> .
Material necesario	Terminal, Python 3.13, venv del Tema07, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en <code>~/Curso_Python/Tema07</code> .

Laboratorio 1. Excepciones básicas y errores frecuentes

1. Actualizar el repositorio, abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
cd ~/Curso_Python
git pull origin main
source .venv/bin/activate
python --version
code .
```

2. Abrir el fichero `Tema07/src/01_excepciones_basicas.py`. Revisar un ejemplo de excepción no capturada. El código queda comentado para que el script completo pueda ejecutarse sin detenerse.

```
# Este código generaría ValueError porque "ssh" no puede convertirse a entero.
# Se deja comentado para que el script completo pueda ejecutarse.
#
# puerto = "ssh"
# numero = int(puerto)
# print(numero)

print("Ejemplo no capturado revisado en comentarios.")
```

3. Aplicar `try/except` básico para capturar un `ValueError` al convertir texto no numérico.

```
valor = "ssh"

try:
    puerto = int(valor)
    print("Puerto convertido:", puerto)
except ValueError:
    print(f"No se puede convertir {valor!r} a número entero.")
```

4. Capturar el objeto de excepción con `as` para mostrar un mensaje claro y, si aporta valor, un detalle técnico.

```
valor = "https"

try:
    puerto = int(valor)
    print("Puerto convertido:", puerto)
except ValueError as error:
    print("Puerto no válido.")
    print("Detalle técnico:", error)
```

5. Revisar errores frecuentes al trabajar con diccionarios y listas: `KeyError` e `IndexError`.

```
datos = {"ssh": 22}
servicios = ["ssh", "nginx"]

try:
    print("Puerto de api:", datos["api"])
except KeyError as error:
    print("Clave inexistente en diccionario:", error)

try:
    print("Servicio en posición 5:", servicios[5])
except IndexError as error:
    print("Índice fuera de rango:", error)
```

6. Capturar una división entre cero y después agrupar varias excepciones cuando la respuesta sea la misma.

```
dividendo = 10
divisor = 0

try:
    resultado = dividendo / divisor
    print(resultado)
except ZeroDivisionError:
    print("No se puede dividir por cero.")

datos = ["22", "abc"]
indice = 2

try:
    puerto = int(datos[indice])
    print("Puerto:", puerto)
except (ValueError, IndexError) as error:
    print("No se pudo obtener el puerto.")
    print("Tipo:", type(error).__name__)
    print("Detalle:", error)
```

7. Ejecutar el script completo.

```
python Tema07/src/01_excepciones_basicas.py
```

Laboratorio 2. Orden de captura, else y finally

1. Abrir el fichero Tema07/src/02_orden_else_finally.py. Revisar la captura específica y el orden de los bloques except.

```
dividendo = "10"
divisor = "2"

# Cambia divisor por "0" para provocar ZeroDivisionError.
# Cambia divisor por "dos" para provocar ValueError.
# Comenta la conversión int(divisor) para provocar TypeError.
try:
    dividendo = int(dividendo)
    divisor = int(divisor)
    resultado = dividendo / divisor
    print("Resultado:", resultado)
except ValueError:
    print("Conversión inválida.")
except TypeError:
    print("El divisor no es numérico.")
except ZeroDivisionError:
    print("No se puede dividir por cero.")
except Exception as error:
    print("Error general no previsto:", type(error).__name__, error)
```

2. Comprobar que la captura genérica debe quedar al final, después de los errores específicos.

```
valor = "abc"

try:
    numero = int(valor)
    print(numero)
except ValueError:
    print("ValueError capturado de forma específica.")
except Exception:
    print("Captura general.")
```

3. Usar else para separar el camino correcto de la gestión del error.

```
valor = "443"

try:
    puerto = int(valor)
except ValueError:
    print("Puerto no válido.")
else:
    print(f"Puerto aceptado: {puerto}")
```

4. Usar `finally` para ejecutar limpieza final, haya error o no.

```

valor = "22"
conexion_abierta = False

try:
    conexion_abierta = True
    print("Conexión abierta:", conexion_abierta)
    puerto = int(valor)
    print("Puerto convertido:", puerto)
except ValueError:
    print("Puerto no válido.")
else:
    print("Operación terminada correctamente.")
finally:
    conexion_abierta = False
    print("Conexión abierta:", conexion_abierta)

```

5. Probar un caso con error y comprobar que `finally` se ejecuta igualmente.

```

valor = "ssh"
recurso_abierto = False

try:
    recurso_abierto = True
    print("Recurso abierto:", recurso_abierto)
    puerto = int(valor)
    print("Puerto convertido:", puerto)
except ValueError as error:
    print("Error al convertir el puerto:", error)
finally:
    recurso_abierto = False
    print("Recurso abierto:", recurso_abierto)

```

6. Ejecutar el script completo.

```
python Tema07/src/02_orden_else_finally.py
```

Laboratorio 3. Validación, EAFP y raise

1. Abrir el fichero `Tema07/src/03_validacion_raise.py`. Comparar el enfoque LBYL, donde se comprueba antes de operar.

```

puertos = {"ssh": 22, "nginx": 443}
servicio = "api"

if servicio in puertos:
    print(puertos[servicio])
else:
    print("Servicio no registrado.")

```

2. Comparar con el enfoque EAFP, donde se intenta la operación y se captura el fallo si aparece.

```
try:
    print(puertos[servicio])
except KeyError:
    print("Servicio no registrado.")
```

3. Usar get () como alternativa de acceso seguro cuando se quiere un valor por defecto.

```
print(puertos.get(servicio, "Servicio no registrado."))
```

4. Lanzar una excepción con raise cuando una función recibe un dato fuera de rango.

```
def validar_puerto(puerto):
    """
    Valida que un puerto esté en el rango permitido.

    Raises:
        ValueError: si el puerto está fuera de rango.
    """
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return puerto

print(validar_puerto(443))

# Descomenta para comprobar el error:
# print(validar_puerto(70000))
```

5. Validar argumentos dentro de una función que construye endpoints.

```
def crear_endpoint(servicio: str, puerto: int) -> str:
    """
    Construye un endpoint servicio:puerto.

    Raises:
        ValueError: si el servicio está vacío o el puerto está fuera de rango.
    """
    if not servicio:
        raise ValueError("Servicio vacío")
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return f"{servicio}:{puerto}"

print(crear_endpoint("ssh", 22))

# Descomenta para comprobar los errores:
# print(crear_endpoint("", 22))
# print(crear_endpoint("ssh", 70000))
```

6. Procesar una colección separando valores válidos y rechazados, sin detener todo el proceso por un único dato defectuoso.

```
datos = ["22", "ssh", "443", "70000"]
validos = []
rechazados = []
```

```

for valor in datos:
    try:
        validos.append(validar_puerto(int(valor)))
    except ValueError as error:
        rechazados.append((valor, str(error)))

print("Válidos:", validos)
print("Rechazados:", rechazados)

```

7. Ejecutar el script completo.

```
python Tema07/src/03_validacion_raise.py
```

Laboratorio 4. Relanzamiento, excepciones personalizadas y traceback

1. Abrir el fichero `Tema07/src/04_relanzar_personalizadas_traceback.py`. Revisar una función que captura un error para añadir contexto y después lo relanza con `raise`.

```

def convertir_puerto(valor):
    """
    Convierte un texto a entero.

    Si falla, añade contexto y relanza la excepción original.
    """
    try:
        return int(valor)
    except ValueError:
        print(f"Fallo al convertir el puerto: {valor}")
        raise

print(convertir_puerto("22"))

try:
    convertir_puerto("ssh")
except ValueError as error:
    print("El llamador recibe el error relanzado:", error)

```

2. Crear una excepción personalizada para representar un error propio del dominio del programa.

```

class ServicioNoDisponibleError(Exception):
    """Error propio del dominio: un servicio esperado no está disponible."""
    pass

def comprobar_servicio(nombre, activo):
    """
    Comprueba si un servicio está activo.

    Raises:
        ServicioNoDisponibleError: si el servicio no está activo.
    """
    if not activo:
        raise ServicioNoDisponibleError(f"{nombre} no está activo")
    return "OK"

```

```
try:
    estado = comprobar_servicio("api", False)
except ServicioNoDisponibleError as error:
    print("Incidencia de servicio.")
    print("Detalle:", error)
else:
    print(estado)
```

3. Usar traceback para imprimir información técnica durante el diagnóstico.

```
import traceback

try:
    int("ssh")
except ValueError as error:
    print("Error controlado:", error)
    print("Traceback técnico:")
    traceback.print_exc()
```

4. Documentar en el docstring qué errores relevantes puede lanzar una función.

```
def obtener_puerto(config, servicio):
    """Devuelve el puerto de un servicio.

    Args:
        config: diccionario con servicios y puertos.
        servicio: nombre del servicio buscado.

    Returns:
        Puerto asociado al servicio.

    Raises:
        KeyError: si el servicio no existe.
    """
    return config[servicio]

puertos = {"ssh": 22}

print(obtener_puerto.__doc__)
print("Puerto ssh:", obtener_puerto(puertos, "ssh"))

try:
    print(obtener_puerto(puertos, "api"))
except KeyError as error:
    print("Servicio no definido en configuración:", error)
```

5. Ejecutar el script completo. Observar que `traceback.print_exc()` muestra una traza técnica, aunque el error esté controlado.

```
python Tema07/src/04_relanzar_personalizadas_traceback.py
```

Laboratorio 5. Ejercicio integrado de gestión de errores

1. Abrir el fichero `Tema07/src/05_ejercicio_integrado_gestion_errores.py`. Revisar las líneas de entrada con registros correctos, puertos no válidos, servicios detenidos y líneas mal formadas.

```
lineas = [
    "ssh;22;activo",
    "nginx;443;activo",
    "api;no_numero;activo",
    "backup;70000;activo",
    "rdp;3389;detenido",
    "malformada",
]

for linea in lineas:
    print(linea)
```

2. Definir una excepción personalizada para servicios detenidos.

```
class ServicioDetenidoError(Exception):
    """Error de dominio: el servicio está registrado, pero no está activo."""
    pass

print("Excepción personalizada definida.")
```

3. Crear funciones de validación e interpretación de líneas. Las funciones lanzan `ValueError` si el formato o el puerto son incorrectos.

```
def validar_puerto(puerto):
    """Valida el rango de un puerto."""
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return puerto

def interpretar_linea(linea):
    """
    Convierte una línea de inventario en un diccionario.

    Raises:
        ValueError: si el formato no es correcto o el puerto no es válido.
    """
    partes = linea.split(";")

    if len(partes) != 3:
        raise ValueError("Formato incorrecto")

    nombre, puerto_txt, estado = partes
    nombre = nombre.strip().lower()
    estado = estado.strip().lower()

    puerto = validar_puerto(int(puerto_txt))

    return {
        "nombre": nombre,
        "puerto": puerto,
        "activo": estado == "activo",
    }
```

4. Crear una función que comprueba disponibilidad y lanza la excepción personalizada si el servicio está detenido.

```
def comprobar_disponible(servicio):
    """
    Comprueba si un servicio está activo.

    Raises:
        ServicioDetenidoError: si el servicio está detenido.
    """
    if not servicio["activo"]:
        raise ServicioDetenidoError(f'{servicio["nombre"]} está detenido')
    return servicio
```

5. Procesar los registros con try/except/else/finally. Los errores se separan por categoría y el proceso continúa.

```
servicios_validos = []
rechazados = []
servicios_detenidos = []

for linea in lineas:
    recurso_abierto = False

    try:
        recurso_abierto = True
        servicio = interpretar_linea(linea)
        servicio = comprobar_disponible(servicio)

    except ServicioDetenidoError as error:
        servicios_detenidos.append((linea, str(error)))

    except ValueError as error:
        rechazados.append((linea, str(error)))

    else:
        servicios_validos.append(servicio)

    finally:
        recurso_abierto = False

print("Servicios válidos:", servicios_validos)
print("Servicios detenidos:", servicios_detenidos)
print("Registros rechazados:", rechazados)
```

6. Generar el reporte final de servicios válidos y contadores de resultado.

```
for servicio in servicios_validos:
    print(f'{servicio["nombre"]}:{servicio["puerto"]} -> OK')

print("Total válidos:", len(servicios_validos))
print("Total detenidos:", len(servicios_detenidos))
print("Total rechazados:", len(rechazados))
```

7. Ejecutar el script completo.

```
python Tema07/src/05_ejercicio_integrado_gestion_errores.py
```

Ejercicios propuestos

1. Crear un script `Tema07/src/validar_puerto_usuario.py` que reciba varios valores de puerto definidos en una lista y separe válidos y rechazados.
2. Crear un script `Tema07/src/acceso_configuracion.py` con un diccionario de servicios y puertos. Probar acceso con clave existente, `get()` y captura de `KeyError`.
3. Crear un script `Tema07/src/division_controlada.py` que divida varios pares de números y capture `ZeroDivisionError` y `ValueError`.
4. Crear un script `Tema07/src/validar_endpoint.py` con una función `crear_endpoint()` que lance `ValueError` si el servicio está vacío o el puerto está fuera de rango.
5. Crear un script `Tema07/src/error_personalizado_backup.py` con una excepción `BackupNoDisponibleError` y una función que la lance cuando un backup esté marcado como fallido.
6. Crear un script `Tema07/src/procesar_registros.py` que procese una lista de líneas `nombre;puerto`, separando registros correctos e incorrectos sin detener el programa.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno virtual no está activado.	Ejecutar <code>source .venv/bin/activate</code> desde <code>~/Curso_Python</code> .
El script no se encuentra	La terminal no está situada en el directorio del tema.	Ejecutar <code>cd ~/Curso_Python</code> y comprobar <code>ls -la</code> .
<code>ValueError</code> al convertir con <code>int()</code>	El texto no representa un número entero válido.	Revisar el dato de entrada o capturar <code>ValueError</code> para informar del problema.
<code>KeyError</code> al leer un diccionario	La clave solicitada no existe.	Comprobar con <code>in</code> , usar <code>get()</code> o capturar <code>KeyError</code> si el fallo es esperable.
<code>IndexError</code> al leer una lista	El índice solicitado está fuera del rango de la colección.	Comprobar <code>len()</code> y recordar que el primer índice es 0.
<code>ZeroDivisionError</code> en una operación	El divisor vale 0.	Validar el divisor antes de dividir o capturar <code>ZeroDivisionError</code> si puede ocurrir.
<code>Exception</code> oculta errores específicos	El bloque <code>except Exception</code> está antes que otros <code>except</code> o es demasiado amplio.	Colocar los <code>except</code> específicos primero y usar <code>Exception</code> solo con un motivo claro.

<code>finally</code> se ejecuta aunque hubo error	Ese es el comportamiento normal de <code>finally</code> .	Usar <code>finally</code> para limpieza final, no para el camino correcto del programa.
<code>traceback.print_exc()</code> muestra una traza, aunque el error está capturado	<code>traceback</code> imprime la información técnica del error actual.	Usarlo para diagnóstico; en producción normalmente registrar el detalle y mostrar mensajes controlados.